EAST WEST UNIVERSITY

PROJECT REPORT

Course: CSE360 (Sec:01)

Semester: Summer-2026

Project Number: 11

IMPLEMENT A CACHE MANAGEMENT POLICY TO REDUCE CACHE POLLUTION AND THRASHING IN C WITH A FIXED SIZE CACHE.

Submitted by

Name	ID	Roll
Sadia Afrin Suchi	2023-3-60-329	27
Nasheba Afrin Ruba	2023-3-60-462	37
Sadia khan Farhin	2023-3-60-539	43
Md Abdus Salam Fahad	2023-3-60-547	44

Submitted to

Dr. Md. Nawab Yousuf Ali

Professor, Department of Computer Science and Engineering East West University

Date of Submission: 18/08/2026

Table Of Contents

Serial	Contents	Page start
1	**Title**	1
2	**Objective**	3
3	**Theory**	3-4
4	**Design**	4
5	**Implementation**	4-7
6	**Debugging-Test-run**	8-10
7	**Results analysis**	10-11
8	**Future Improvements**	11-12
9	**Conclusion**	12
10	**Bibliography**	12

Objective

The objective of this project is to design and implement a cache management policy in C that reduces cache pollution and thrashing within a fixed size cache. Cache memory is limited in size, so whenever it becomes full, the system must decide which existing entry to remove to make room for a newly requested one. A naive policy such as plain FIFO (First In First Out) only considers the order in which pages arrived, and completely ignores whether a page is still being actively used. This can cause a frequently used ("hot") page to be evicted just because it happens to be the oldest in the queue - a problem known as cache pollution. When the cache repeatedly evicts and reloads data without any real gain in performance, it is called thrashing.

In this project, the Second Chance (Clock) algorithm is implemented as an improved replacement policy. It extends plain FIFO with a single reference bit per cache slot, giving recently used pages a "second chance" before they are evicted. To evaluate its effectiveness, the Second Chance algorithm is compared against a plain FIFO implementation under identical conditions - same fixed cache size and the same input reference sequence - so that any difference in performance can be attributed directly to the replacement policy itself.

Theory

A cache is a small, fast storage area that keeps a limited number of frequently accessed items so that they can be retrieved quickly without going back to a much larger and slower source. Because a cache has a fixed capacity, a replacement policy is required to decide which entry to discard whenever a new item must be inserted into a full cache.

Cache Pollution

Cache pollution occurs when infrequently used or one-time-access data occupies cache space that would otherwise be used by frequently accessed ("hot") data. This forces hot data out of the cache prematurely, increasing the miss rate for future requests.

Cache Thrashing

Thrashing occurs when the working set (the set of items actually being used) is larger than the cache can hold, or when the replacement policy repeatedly evicts data that will be needed again very soon. The cache then spends most of its time reloading data rather than serving hits, and system performance degrades.

Second Chance (Clock) Algorithm

The Second Chance algorithm improves on FIFO by attaching one reference bit (refBit) to every cache slot. A circular pointer, called the "hand", scans the cache slots in order whenever a replacement is needed:

- If the slot pointed to by hand is empty, the new page is placed there directly.
- If refBit of the current slot is 1 (recently used), it is reset to 0 and the hand moves to the next slot - the page is given a second chance instead of being evicted immediately.

- If refBit of the current slot is 0, that page is evicted and the new page takes its place.
- On a cache hit, the refBit of the matching slot is simply set to 1.

Because a page's refBit is refreshed to 1 every time it is accessed again, pages that are still in active use survive longer in the cache even though the underlying structure is still a circular FIFO queue. This is the mechanism that is expected to reduce cache pollution compared with plain FIFO.

Design

Second Chance (clock) algorithms operate on a fixed size cache represented as a simple array, so no dynamic memory allocation is required. The design keeps three main elements: the cache array (holding page numbers, -1 for an empty slot), a parallel refBit array (used only by Second Chance) and a pointer that tracks where the next replacement should be attempted - a linear front index for FIFO, and a circular hand pointer for Second Chance.

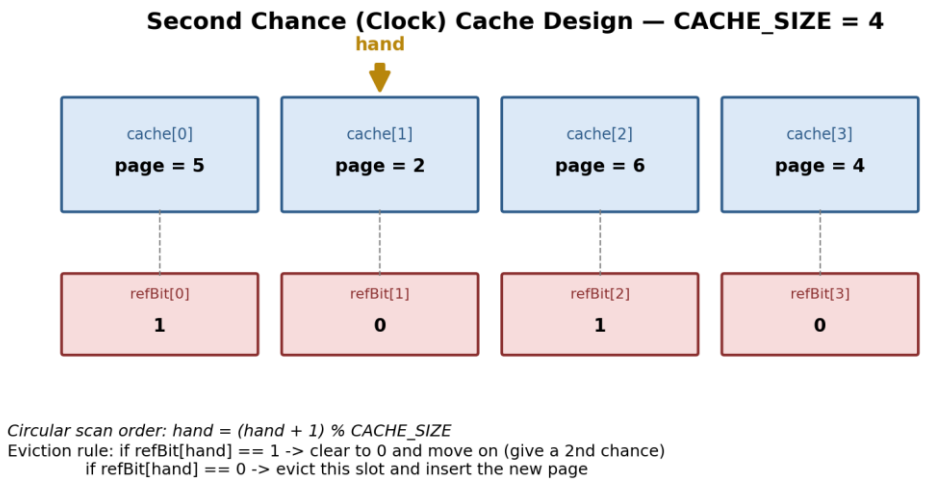


Fig. 1 - Cache array, refBit array, and clock hand used by the Second Chance algorithm.

For every referenced page, the design first performs a linear search of the cache array. If the page is found, it is a HIT (and for Second Chance, its refBit is set to 1). If it is not found, it is a MISS, and the appropriate replacement procedure is triggered to bring the page into the cache.

Implementation

Plain FIFO - C Implementation

The FIFO version keeps a single index, front, which always points to the oldest page in the cache. On every miss, the new page simply overwrites cache[front], and front is advanced circularly. A cache hit does not change the array or the pointer at all - this is exactly what makes FIFO blind to recency of use.

```
#include <stdio.h>
#define CACHE_SIZE 4
```

```
int cache[CACHE_SIZE];
int front = 0;   // points to the oldest page (next to be evicted)

void displayCache()
{
   int i;
   printf("Cache: ");
   for(i = 0; i < CACHE_SIZE; i++)
   {
      if(cache[i] == -1)
         printf("- ");
      else
         printf("%d ", cache[i]);
   }
   printf("\n");
}

int search(int page)
{
   int i;
   for(i = 0; i < CACHE_SIZE; i++)
      if(cache[i] == page)
         return i;
   return -1;
}

void fifoReplace(int page)
{
   int pos = search(page);
   if(pos != -1)
   {
      printf("Page %d -> HIT\n", page);
      return;              // FIFO does not reorder on a hit
   }
   printf("Page %d -> MISS\n", page);
   cache[front] = page;
   front = (front + 1) % CACHE_SIZE;
}

int main()
{
   int i, hits = 0, misses = 0;
   int pages[] = {1,2,3,4,5,2,6,2,7,2,8,2};
   int n = sizeof(pages)/sizeof(pages[0]);

   for(i = 0; i < CACHE_SIZE; i++)
      cache[i] = -1;

   printf("Plain FIFO Cache Management\n\n");
   for(i = 0; i < n; i++)
```

```
    {
        if(search(pages[i]) != -1) hits++; else misses++;
        fifoReplace(pages[i]);
        displayCache();
        printf("\n");
    }
    printf("Hits   : %d\n", hits);
    printf("Misses : %d\n", misses);
    printf("Hit Ratio = %.2f%%\n", (hits * 100.0) / (hits + misses));
    return 0;
}
```

Second Chance (Clock) - C Implementation

The Second Chance version adds the refBit array and replaces the single front pointer with a circular hand pointer. The replacement loop keeps advancing the hand and clearing reference bits until it finds a slot whose refBit is already 0 (or an empty slot), which is where the new page is finally placed.

```
#include <stdio.h>
#define CACHE_SIZE 4
int cache[CACHE_SIZE];
int refBit[CACHE_SIZE];
int hand = 0;

void displayCache()
{
    int i;
    printf("Cache: ");
    for(i = 0; i < CACHE_SIZE; i++)
    {
        if(cache[i] == -1)
            printf("- ");
        else
            printf("%d ", cache[i]);
    }
    printf("\n");
}

int search(int page)
{
    int i;
    for(i = 0; i < CACHE_SIZE; i++)
        if(cache[i] == page)
            return i;
    return -1;
}

void secondChance(int page)
{
```

```
    int pos = search(page);
    if(pos != -1)
    {
        printf("Page %d -> HIT\n", page);
        refBit[pos] = 1;
        return;
    }
    printf("Page %d -> MISS\n", page);
    while(1)
    {
        if(cache[hand] == -1)
        {
            cache[hand] = page; refBit[hand] = 1;
            hand = (hand + 1) % CACHE_SIZE;
            break;
        }
        if(refBit[hand] == 0)
        {
            cache[hand] = page; refBit[hand] = 1;
            hand = (hand + 1) % CACHE_SIZE;
            break;
        }
        refBit[hand] = 0;
        hand = (hand + 1) % CACHE_SIZE;
    }
}

int main()
{
    int i, hits = 0, misses = 0;
    int pages[] = {1,2,3,4,5,2,6,2,7,2,8,2};
    int n = sizeof(pages)/sizeof(pages[0]);

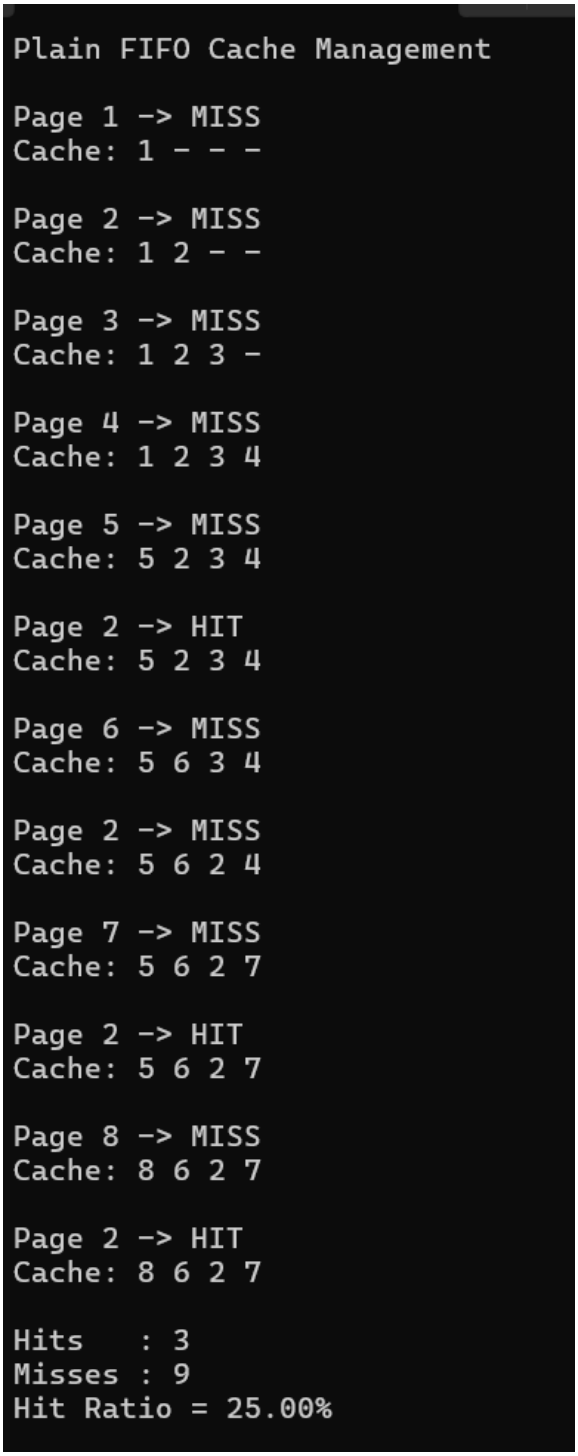
    for(i = 0; i < CACHE_SIZE; i++) { cache[i] = -1; refBit[i] = 0; }

    printf("Second Chance Cache Management\n\n");
    for(i = 0; i < n; i++)
    {
        if(search(pages[i]) != -1) hits++; else misses++;
        secondChance(pages[i]);
        displayCache();
        printf("\n");
    }
    printf("Hits   : %d\n", hits);
    printf("Misses : %d\n", misses);
    printf("Hit Ratio = %.2f%%\n", (hits * 100.0) / (hits + misses));
    return 0;
}
```

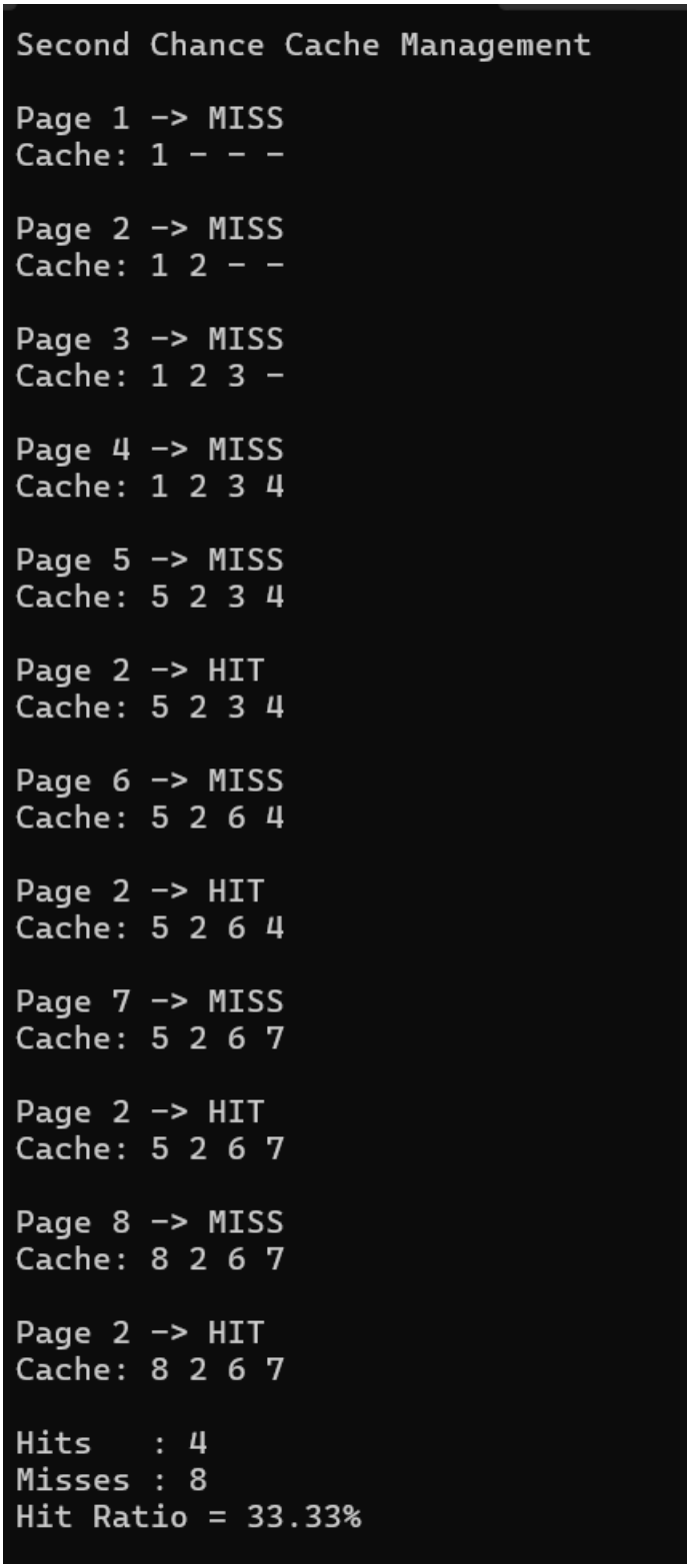
Debugging - Test Run

Both programs were compiled with gcc and executed on the same reference sequence {1, 2, 3, 4, 5, 2, 6, 2, 7, 2, 8, 2} with CACHE_SIZE = 4, so that the two policies are compared under identical conditions. Page 2 is deliberately referenced four times to represent a "hot" page, while the remaining pages (1, 3, 4, 5, 6, 7, 8) each appear only once, representing one-time, potentially polluting references.

FIFO Output



Second Chance Output



Both outputs were produced by compiling and running the exact source code shown in the Implementation section , confirming that the programs work correctly and match the expected step-by-step behaviour of each algorithm.

Results Analysis

Metric	Plain FIFO	Second Chance
Cache Size	4	4
Total References	12	12
Hits	3	4
Misses	9	8
Hit Ratio	25.00%	33.33%

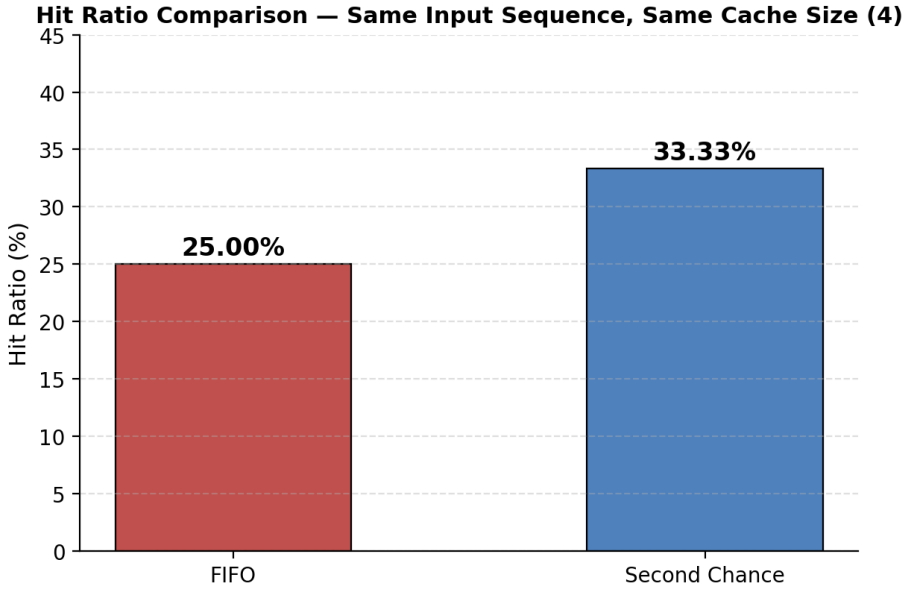


Fig. 2 - Hit ratio comparison between Plain FIFO and Second Chance on the same input.

The key difference between the two runs happens at the reference "Page 6". At that point, page 2 (the hot page) is sitting in the cache with its refBit already set to 1 from an earlier hit. Plain FIFO ignores this entirely and evicts whichever page is oldest in the queue - in this case page 2 itself - producing an avoidable miss the next time page 2 is requested. Second Chance, on the other hand, checks refBit before evicting: because page 2's refBit is 1, it is spared and a colder page is evicted in its place instead.

This single difference is responsible for the entire gap in performance: Second Chance records one additional hit (4 instead of 3) and one fewer miss (8 instead of 9), raising the hit ratio from 25.00% to 33.33% on an identical input sequence and identical cache size. Since the cache capacity and the reference sequence were kept exactly the same for both runs, this improvement can be attributed directly to the refBit mechanism - which is precisely the feature designed to reduce cache pollution and protect frequently used pages from being displaced by one-time, polluting references.

Future Improvements

The Second Chance algorithm is a meaningful improvement over plain FIFO, but it is still a relatively simple policy. Several directions could improve it further:

- Segmented LRU (SLRU): split the cache into a probationary segment and a protected segment, so that only pages accessed more than once are promoted to the protected area, giving stronger protection against one-time polluting scans.
- Frequency-aware policies (LFU with aging): track how many times a page has been used, with older counts periodically decayed, so long-unused "cold" pages are not able to permanently hold a cache slot on the strength of a single old burst of use.
- Adaptive Replacement Cache (ARC): dynamically balance between recency and frequency based on the observed workload, instead of using a single fixed rule.
- Multiple reference bits or an access counter per slot, instead of a single bit, to more finely rank how "hot" each page currently is.
- Testing with larger, more realistic and longer reference traces (e.g. simulated web-cache or database access logs) to evaluate the policies beyond small hand-designed sequences.

Conclusion

This project implemented a fixed size cache in C and evaluated the Second Chance (Clock) replacement algorithm as a way to reduce cache pollution and thrashing, comparing it against a plain FIFO baseline under identical conditions. The results confirm the underlying theory: by attaching a single reference bit to each cache slot, Second Chance is able to recognise and protect frequently used ("hot") pages that plain FIFO would blindly evict simply because they were the oldest entries in the queue. On the test sequence used in this report, this improved the hit ratio from 25.00% (FIFO) to 33.33% (Second Chance) with no change to the cache size or the input sequence, showing that the replacement policy alone was responsible for the improvement. This supports the project's central goal: a low-overhead, easy-to-implement modification to a classic FIFO cache can meaningfully reduce cache pollution.

Bibliography

- GeeksforGeeks - Page Replacement Algorithms in Operating Systems: https://www.geeksforgeeks.org/page-replacement-algorithms-in-operating-systems/
- Wikipedia - Cache replacement policies: https://en.wikipedia.org/wiki/Cache_replacement_policies
- Wikipedia - Page replacement algorithm (Second-chance / Clock): https://en.wikipedia.org/wiki/Page_replacement_algorithm
- GeeksforGeeks - Cache memory in Computer Organization: https://www.geeksforgeeks.org/cache-memory-in-computer-organization/